

TECHNICAL INTERVIEW GUIDE

AI Harness Engineering Interview Preparation Handbook

Runtime, Control Layer, Guardrails, MCP,
Evals, and Observability for Production AI Agents

A practitioner's guide to the second half of every AI agent



AI HARNESS ENGINEERING

Interview Preparation Handbook

*Runtime, Control Layer, Guardrails, MCP,
Evals, and Observability for Production AI Agents*

A practitioner's guide
to the second half of every AI agent

AI Engineering Insider

2026 Edition

AI Harness Engineering Interview Preparation Handbook

Runtime, Control Layer, Guardrails, MCP, Evals, and Observability for Production AI Agents.

By AI Engineering Insider.

Edition 1.0, 2026.

This book is a teaching reference. The field of AI agent engineering moves faster than any printed source. Trust primary sources — lab engineering blogs, the OWASP LLM Top 10, the MCP specification, and the papers cited in the chapters that introduce them — over any secondary summary including this one. When in doubt, run the experiment and write the eval.

Code examples are illustrative and not intended to be copy-pasted into production without review.

*For the engineers on call at 3 a.m.
when the agent did something nobody predicted.*

Contents

Preface	ix
I Foundations	1
1 What Is AI Harness Engineering?	2
1.1 A one-sentence definition	2
1.2 The mental model: Agent equals Model plus Harness	3
1.3 The four-layer view of AI software	4
1.4 The equestrian analogy	4
1.5 Where production AI agents actually live	5
1.6 Harness the platform, harness the concept	6
1.7 The shift from prompt engineering to harness engineering	6
2 Core Agent Concepts	8
2.1 The agent loop	8
2.2 State and continuation	9
2.3 Tools and function schemas	10
2.4 Handoffs and specialist sub-agents	11
2.5 Single-agent versus multi-agent: a decision framework	11
2.6 When not to use an agent at all	12
2.7 Canonical loop architectures	12
2.8 Agent looping failure	14
3 Anatomy of a Harness — The Seven Layers	16
3.1 Why seven	16
3.2 Layer 1: Instruction	17
3.3 Layer 2: Tools	17
3.4 Layer 3: Memory and retrieval	17
3.5 Layer 4: Execution	18
3.6 Layer 5: Policy and approval	18
3.7 Layer 6: Observability	19
3.8 Layer 7: Evaluation	19
3.9 The dependency graph	19
3.10 Which layer to investigate first	20
3.11 Which layer to build first	20
3.12 A worked example: applying the seven layers	21

Part I System Design Prompt	23
II Context and Instruction Engineering	26
4 Instruction Engineering for Agents	27
4.1 The three tiers of agent context	27
4.2 What belongs in the system prompt	27
4.3 Structured outputs	28
4.4 Constraint design	29
4.5 Failure-aware prompting	30
4.6 Prompt versioning	30
4.7 Prompt libraries and policy variables	31
4.8 Spotighting: untrusted content marking	31
4.9 Context window economics	32
5 Skills and Reusable Behaviors	34
5.1 From prompt strings to Skills	34
5.2 The anatomy of a Skill	34
5.3 A taxonomy of Skills	36
5.4 Editor Skills versus pipeline Skills	36
5.5 Skill discovery and selection	37
5.6 Testing Skills: regression suites for prompt templates	37
5.7 Skill drift and maintenance	38
5.8 A/B testing Skill versions	38
5.9 Skills versus tools: a distinction worth learning	39
5.10 Harness Skills: the canonical catalogue	39
Part II System Design Prompt	41
III Tools, Actions, and MCP	44
6 Tool Calling and Safe Action Design	45
6.1 The trust ladder	45
6.2 Function schema design	46
6.3 Input validation beyond the schema	47
6.4 Read versus write: the separation principle	48
6.5 Error messages for LLM consumption	48
6.6 Retries, idempotency, and at-most-once semantics	49
6.7 Human approval gates	50
6.8 Dry-run mode	51
6.9 Permission checks: the principal problem	51
6.10 Tool result formatting	52
6.11 A worked example: the Pipeline Mutation Tool	52
7 MCP — The Agent Integration Standard	54
7.1 What MCP is, in one sentence	54
7.2 Why a protocol matters	54

7.3	Clients, servers, transports	55
7.4	The three primitives	56
7.5	Discovery and capability boundaries	57
7.6	Authentication and authorization	58
7.7	Composition: connecting multiple MCP servers	58
7.8	Security pitfalls	59
7.9	When to use MCP versus alternatives	60
7.10	A worked example: designing an MCP server for a CI/CD platform	60
8	Harness MCP Server and Platform Integration	63
8.1	Why a platform-native MCP server matters	63
8.2	What the Harness MCP Server exposes	63
8.3	Accessing Harness resources through an agent	65
8.4	The authorization model	65
8.5	Audit trails for MCP-driven operations	66
8.6	Designing governed agent workflows on top of Harness	67
8.7	Extending the Harness MCP Server	68
8.8	A worked example: natural-language pipeline creation, end to end	68
	Part III System Design Prompt	71
IV	Memory, State, and Retrieval	75
9	State, Memory, and Context Windows	76
9.1	Three kinds of memory, three kinds of problem	76
9.2	Scratch memory: the in-context surface	77
9.3	Episodic memory: learning across runs	78
9.4	Semantic memory: the domain model	79
9.5	Memory decay and re-validation	79
9.6	Memory poisoning and its defences	80
9.7	When memory makes the agent worse	81
9.8	A worked example: memory for a long-running coding agent	81
10	Retrieval and Grounded Reasoning	83
10.1	Retrieval as a first-class system	83
10.2	Chunking	83
10.3	Embedding choice	84
10.4	Hybrid retrieval: why BM25 is not obsolete	85
10.5	Reranking	85
10.6	Source ranking: beyond relevance	86
10.7	Citations and provenance	86
10.8	Trust boundaries in retrieval	86
10.9	Stale data detection	87
10.10	RAG for enterprise agents	87
10.11	The repository-as-source-of-truth pattern	87
10.12	A worked example: the incident-investigation retrieval stack	88

Part IV System Design Prompt	90
V Sandboxed Execution Environments	94
11 Isolation, Sandboxing, and Fast Execution	95
11.1 Why trust boundaries exist	95
11.2 The trust ladder	95
11.3 Why default Docker isn't enough	96
11.4 gVisor and Firecracker: the purpose-built rungs	97
11.5 Per-task worktree isolation	97
11.6 Network egress policies	98
11.7 Filesystem controls	98
11.8 Credential scoping	99
11.9 Fast-start patterns	99
11.10 Managed sandbox services	99
11.11A worked example: the coding-agent sandbox	100
Part V System Design Prompt	102
VI Verification Loops and Quality Engineering	106
12 Guides, Sensors, and Self-Correction	107
12.1 The reframe	107
12.2 Why you need both	108
12.3 Computational versus inferential controls	108
12.4 Writing checker messages for LLM consumption	109
12.5 The approved-fixtures pattern	110
12.6 Golden principles	110
12.7 The recursive self-correction loop	111
12.8 Validating the inferential controls	111
12.9 The missing-sensor test	112
12.10A worked example: the pipeline-generation verification layer	112
Part VI System Design Prompt	114
VII Evaluation and Benchmarking	117
13 How to Evaluate an Agent	118
13.1 Why evaluation is engineering, not QA	118
13.2 What to evaluate	118
13.3 Golden datasets	120
13.4 LLM-as-judge	121
13.5 Regression testing in CI	122
13.6 Statistical rigor	123
13.7 Eval-as-a-service	124
13.8 Benchmarks to know by name and by mechanism	124

13.9	The eval-debugging discipline	125
14	Failure Modes and Reliability Engineering	127
14.1	A taxonomy of agent failures	127
14.2	Diagnosing from traces	128
14.3	Recovery strategies	128
14.4	Eval-driven debugging	129
14.5	Chaos engineering for agents	130
14.6	Blast radius minimisation	130
14.7	Partially-applied state	131
14.8	The on-call runbook	131
14.9	Reliability metrics that matter	132
Part VII	System Design Prompt	134
VIII	Safety, Guardrails, and Governance	138
15	Agent Safety and Guardrails	139
15.1	What guardrails actually are	139
15.2	Input, inline, and output guardrails	139
15.3	Prompt injection: the central threat	141
15.4	PII and data-handling	142
15.5	Secret handling	143
15.6	Policy engines	143
15.7	Refusal, and how to do it well	143
15.8	Content filtering	144
15.9	Regional and regulatory compliance	144
15.10A	worked example: guardrail stack for an enterprise support agent	145
16	Secure Agent Operations and Enterprise Governance	147
16.1	The governance perimeter	147
16.2	Identity and authentication	148
16.3	RBAC and delegated scopes	148
16.4	Change management integration	149
16.5	Audit and SIEM integration	149
16.6	Approval workflows at scale	150
16.7	Data residency and egress	151
16.8	Vendor risk and third-party models	151
16.9	Incident response integration	151
16.10	The trust handoff: enterprise onboarding	152
16.11	The governance scorecard	152
Part VIII	System Design Prompt	154
IX	Observability, Runtime, and Cost	157
17	Tracing and Observability	158

17.1 Why traditional observability isn't enough	158
17.2 GenAI OpenTelemetry semantics	158
17.3 Agent run as a trace	159
17.4 Cost and budget visibility	160
17.5 Prompt and completion capture	160
17.6 Anomaly detection on traces	161
17.7 Feedback signal integration	161
17.8 Sampling and retention	162
17.9 A worked example: the observability stack for a 40-Skill platform	162
18 Runtime Engineering and Cost Control	165
18.1 Runtime engineering as a discipline	165
18.2 Model routing	165
18.3 Prompt caching	166
18.4 Batch APIs	167
18.5 Token budgeting	167
18.6 Concurrency control	167
18.7 Latency optimisation	168
18.8 Caching beyond prompts	169
18.9 Fallback and degraded modes	169
18.10 Operational runbook	169
18.11 The cost model	170
Part IX System Design Prompt	172
X Multi-Agent Systems and Orchestration	175
19 Multi-Agent Design and Durable Orchestration	176
19.1 The single-agent default	176
19.2 Agent composition patterns	177
19.3 Handoff protocols	178
19.4 Shared state versus message passing	179
19.5 Durable orchestration	179
19.6 Coordination failure modes	180
19.7 Observability across agents	181
19.8 A worked example: multi-agent incident triage	181
Part X System Design Prompt	183
Afterword	187
Production Readiness Checklist	190
Glossary	195

Preface

A production AI agent has two halves: the model and the harness. The model is a commodity. The harness is the engineering.

— Core thesis

What this book is about

In 2023, the conversation was about prompts. In 2024, about agents. In 2025, about tools and context. As of 2026, the conversation has consolidated around a single word that the best engineers in the field already use without needing to define: the **harness**.

The harness is everything around the model. It is the tools the agent can call and the schemas that constrain those calls. It is the sandbox the agent runs in and the allowlist that bounds its network egress. It is the guardrails that inspect its inputs and outputs, the memory system that records what it has done, the traces that prove what it did, and the evals that decide whether you let it do it again. It is the policy layer that refuses a write, the approval gate that pauses for a human, and the router that picks a cheaper model when the task is easy. It is, in short, all the engineering.

The model inside the harness will be different in a year. The harness principles in this book will not be.

Why this is a distinct discipline

A decade ago, building a production service meant knowing how to write code, deploy it, monitor it, and secure it. Each of those became a sub-discipline: SRE, DevOps, SecOps, platform engineering. The same fragmentation is happening now with AI agents, and the crystallising discipline — the one that sits above prompt engineering and below application logic — is harness engineering.

A harness engineer is not a prompt whisperer. A prompt whisperer is trying to get one conversation to go well. A harness engineer is building the infrastructure that makes a hundred thousand conversations go well when nobody is watching, when the inputs are adversarial, when the model provider silently updates its weights, and when the cost accountant wants a five-times reduction by Friday.

The skill set is recognisable from distributed systems, but not identical. You will recognise queues, timeouts, retries, idempotency, circuit breakers, and observability. You will also meet some concepts that have no clean analogue in traditional systems: structured-output contracts, tool-call trajectories, prompt injection, memory poisoning, trajectory evaluation, dual-LLM safety archi-

textures, and the entire subgenre of techniques for telling a non-deterministic component to behave deterministically anyway.

Who this book is for

This book is written for five audiences, and the chapters are labelled so that each audience can find its own path quickly.

- **AI engineers** building agentic products — you will live in Parts I through VII.
- **Platform and infrastructure engineers** integrating AI into existing systems — Parts III, V, and IX are the load-bearing ones.
- **SDETs and evaluation specialists** — Parts VI and VII are your home; treat the rest as background context.
- **DevOps and MLOps engineers** running agents in production — Parts IX and X will feel most familiar; Part VIII will surprise you.
- **Security, trust, and safety engineers** — Part VIII is the core; read Parts III and IV to understand the attack surface.
- **Interview candidates** targeting frontier AI labs, foundation model companies, and AI-first platforms — read everything, and do the exercises at the end of each chapter out loud.

How the book is structured

The book is organised into ten Parts that move outward from foundations to production concerns.

Foundations (Parts I and II) — mental models, the seven-layer view of a harness, and the instruction layer that anchors every agent.

Tools, integration, and data (Parts III and IV) — safe action design, the Model Context Protocol, the Harness MCP Server, memory systems, and retrieval pipelines that ground agents in reality.

Execution and verification (Parts V and VI) — sandboxing for arbitrary code execution, then guides and sensors that close the verification loop around model output.

Quality, safety, and operations (Parts VII through IX) — evaluation as engineering, failure-mode taxonomies, the guardrail and governance machinery that turns a working agent into a deployable product, and the observability and runtime engineering that keeps it running at production scale.

Composition (Part X) — multi-agent design, durable orchestration, and the disciplines for coordinating agents without making coordination the bottleneck.

Each Part closes with a System Design Prompt — a 45-minute scope problem worked end-to-end, with a model answer and a section explaining what strong answers have in common. Each chapter closes with an Interview Focus box of 5–7 questions and the framing of a strong answer to each. The appendices collect a production readiness checklist and a glossary of the terms used throughout the book.

How to read this book

Do not read it passively.

Each chapter ends with an *Interview Focus* box containing the questions that recur in real interviews for this material. After you finish a chapter, close the book and answer those questions out loud, as if a panel were across the table. You will notice quickly that reading a concept and articulating it back are different skills, and interviews measure the second. This book is built around that asymmetry.

Each Part ends with a *System Design Prompt* — a 45-minute scope problem that exercises every layer the Part has covered. Treat them as real: sketch a diagram, timebox yourself, state your assumptions aloud, and write down what you would build first, second, and third. The prompt rewards design judgment more than comprehensive coverage.

Read the Parts in order on the first pass. The chapters cross-reference heavily and the framing established in Part I (the seven-layer model, the agent loop, the trust ladder) carries through every later chapter. On a second pass, jump to the Parts most relevant to your role.

The three skills interviewers actually test

Across dozens of harness-engineering interview loops, three skills predict the signal more than any amount of surface knowledge.

Design judgment. The ability to choose, under time pressure, among several reasonable architectures and defend the choice with reference to trust model, scale, cost, and failure modes. This is what the system design round measures. It is almost never about knowing one correct answer. It is about demonstrating that you have internalised the tradeoffs.

Operational instinct. The ability to reason about what an agent actually does in production, minute to minute, under load, under adversarial input, when the model provider has a bad hour. This is what the deep-dive round measures. It surfaces in questions like “*your eval says 85% success but production shows 60%; debug it.*”

Failure fluency. The ability to name the ways a system breaks before it breaks, to reason about blast radius, and to design recovery paths that respect the reversibility of actions. This is what the most senior engineers in the room are listening for. If you have been on call for an agent, you have it. If you have not, the chaos-engineering and failure-taxonomy chapters are your shortcut.

A note on the field

This field moves faster than any book can keep up with. Specific tools will be replaced. Specific benchmarks will be saturated. Specific model providers will be merged or eclipsed. The *shape* of the problem — an intelligent but unreliable component that must be wrapped in engineered structure to be trustworthy — will not change in the next decade, and probably not in the next two.

Learn the shape. The tooling will follow.

— *AI Engineering Insider and Lamhot Siagian*
2026

PART I

Foundations

What Is AI Harness Engineering?

The horse is the model. The harness is everything that turns the horse into useful work.

— Operating principle

1.1 A one-sentence definition

AI harness engineering is the discipline of designing, building, and operating the runtime and control layer that wraps a language model so that its outputs can be trusted, audited, and composed into production software.

That sentence carries most of the book. Unpack it.

Discipline — not a checklist, not a library, not a vibe. A way of thinking about the problem with a shared vocabulary and shared failure modes. Every mature engineering discipline has its own debugging instincts, its own tradeoff grammar, and its own legible artefacts; harness engineering is acquiring all three as the field matures.

Runtime and control layer — the harness is code that runs, not prose that is written. A system prompt is an *input* to the harness. The harness itself is the tool registry that dispatches calls, the sandbox that isolates execution, the guardrails that intercept outputs, the tracer that records everything, and the evaluator that decides whether this run was acceptable.

Wraps a language model — the model sits at the centre. Around it, in concentric shells, are the instruction layer, tools, memory, execution environment, policy, observability, and evaluation. None of these shells *is* the model. All of them, together, are what turns a model into an agent.

Outputs can be trusted, audited, and composed — the three properties that distinguish a demo from a production system. Trust is a guarantee about the next run. Audit is a record of the last run. Composition is the property that lets you plug this agent into a larger system without the larger system having to re-validate every assumption.

Key Insight

If you can explain the definition above to a senior engineer in one minute without referring to any specific tool, framework, or vendor, you have passed the first test of this field. Interviewers open with this kind of definition question because it separates candidates who have absorbed a framework from candidates who have absorbed the concept.

1.2 The mental model: Agent equals Model plus Harness

The single most useful equation in this book is:

$$\text{Agent} = \text{Model} + \text{Harness}$$

Two observations about this equation.

First, it is *additive*. Swap the model and, if the harness is well designed, the rest of the system keeps working. Swap the harness and you effectively have a different product even with the same model. The best evidence of a well-designed harness is that you can upgrade the model with a pull request.

Second, it is *asymmetric in economic terms*. The model is a commodity that you buy from a vendor by the token. The harness is capital that you build, own, and differentiate on. Two companies using the same model will produce different products because their harnesses will differ. This is why frontier labs employ harness engineers as a distinct role and not as a sub-skill of application developers.

1.2.1 Why “a smarter model” does not save you

A tempting thought in early 2024 was: if the model gets smart enough, most of the harness will become unnecessary. The model will just do the right thing.

Two years of production experience have refuted this. A smarter model is better at what it does, but production systems fail on properties that intelligence alone does not provide:

- **Determinism on demand.** Business logic sometimes requires the same input to produce the same output. Smarter models still sample.
- **Auditability.** A post-incident review needs to know what the agent did and why. A smarter model does not log itself.
- **Permission enforcement.** The agent should not be able to exceed the permissions of the user who invoked it. A smarter model does not know what permissions are.
- **Cost bounds.** A task must not cost more than \$1.20 to run. A smarter model may reason for twenty minutes and produce the right answer at the wrong price.
- **Adversarial robustness.** A user, or a malicious web page, will try to redirect the agent. A smarter model is a *more capable* target, not a less tempting one.
- **Composition.** The agent must be callable by other systems and compose cleanly with queues, retries, and timeouts. A smarter model is not a better-composing unit of software.

All six properties are harness-level properties. They are provided by the engineering around the model, not by the model itself. Smarter models raise the ceiling of what agents can do. Harness engineering raises the floor of what you can ship.

Common Trap

In interviews, the failure mode that disqualifies candidates fastest is appealing to *model capability* as the answer to a *systems* question. “We’d use Claude 4.7 which is really good at this” is not an answer to “how do you prevent an infinite loop.” The answer is a step

budget, a trace-driven loop detector, and a tool-call deduplication check — all harness concerns.

1.3 The four-layer view of AI software

To place harness engineering in context, distinguish four layers. Each has its own abstractions, its own vocabulary, and its own failure modes.

1. **Model.** The language model itself. A pre-trained, post-trained, possibly tool-use-fine-tuned neural network accessed via an API. Its interface is `messages in`, `messages out`, possibly with a `tool_calls` field.
2. **Agent.** A loop that uses the model to perceive, plan, act, and iterate. It knows about tools, context, and termination conditions. Its interface is `task in`, `completed-task-or-escalation out`.
3. **Workflow.** A composition of agents and deterministic steps, orchestrated to accomplish a larger business process. It knows about handoffs, approval gates, and durable state. Its interface is `trigger in`, `business-outcome out`.
4. **Harness.** The cross-cutting runtime that all of the above depend on. It knows about tool schemas, sandboxes, guardrails, memory, traces, evals, costs, permissions, and rollbacks. Its interface is not a single API; it is the environment in which the other three layers are *safe to run*.

The model is a component. The agent is a pattern. The workflow is a product. The harness is the platform.

Key Insight

When an interviewer asks “*where does the agent end and the harness begin?*” the answer is: the agent is the loop and its logic; the harness is everything the loop *depends on* to run safely. A tool is defined by the agent but executed by the harness. A system prompt is authored for the agent but versioned and served by the harness. This distinction is worth rehearsing until it is automatic.

1.4 The equestrian analogy

The word *harness* is deliberately chosen. It maps onto an older practice of humans directing powerful non-human systems, and the mapping is tight enough to be useful.

The horse is the model. It is strong, sometimes brilliant, and not your employee. You did not train it, and if you did you did not finish the job. It has moods. It misunderstands instructions. It cannot read the road signs. It will walk into traffic if nothing stops it.

The harness is the engineered apparatus that connects the horse to the cart, the reins to the bit, and the driver to the system. The harness doesn’t make the horse faster. It makes the horse’s power *useful*. Without a harness, a horse is a liability in a city. With a harness, the horse is transportation.

The rider is the human with intent. The rider knows the destination. The rider makes the moral choices. The rider is accountable for where the cart ends up. The rider is not watching the horse's feet — the harness does that.

The analogy is not perfect; no analogy is. But it captures three important properties of the real situation: the model is not the adversary, it is the *asset*; the engineer's job is not to make the model do things but to make its outputs *useful and safe*; and the human remains accountable, always, for the system's effect on the world.

1.5 Where production AI agents actually live

Harness engineering is not a research topic. The patterns in this book were crystallised in response to live production deployments, several of which you will see recur as case studies. The categories that matter most today:

1.5.1 Coding agents

The most mature agent category, because it has a clean verification signal (tests) and a bounded action space (a sandboxed worktree). Coding agents now write, edit, review, and remediate code across files, run tests in isolation, and open pull requests. SWE-bench and related benchmarks have made the capability frontier measurable. The harness engineering challenges are sandboxing at scale, test-driven self-correction, long-horizon context management, and repo-as-source-of-truth patterns.

1.5.2 CI/CD and pipeline automation

Agents that create, modify, and debug CI/CD pipelines via natural language. The Harness DevOps Agent is the canonical example. Because these agents operate on infrastructure-as-code and trigger real deployments, the harness engineering emphasis is on RBAC inheritance, dry-run modes, approval gates, and irreversibility management. The interview questions here skew heavily toward safe-action design.

1.5.3 Support and triage

Customer support agents that classify incoming issues, retrieve relevant documentation, draft responses, and escalate to humans when confidence is low. These agents live on top of knowledge bases and ticketing systems, and the harness engineering emphasis is on grounding (retrieving and citing documentation), policy compliance (what can the agent commit to?), and confidence calibration (when does it escalate?).

1.5.4 Security remediation

Agents that read vulnerability reports, correlate them to code, and open pull requests with fixes. The harness engineering emphasis is severity triage (auto-fix for low-severity, approval gate for high-severity), RBAC scoping, and audit trails. This is a high-stakes category because the agent's action is directly mutative to production code, and the verification signal (did we actually fix the vuln without breaking something else?) is not local.

1.5.5 Document-grounded assistants

Agents that answer questions grounded in a corpus: legal documents, contracts, runbooks, API specs, incident reports. The harness engineering emphasis is retrieval design, citation integrity, access control (users should only retrieve documents they are authorised to see), and the architectural discipline to refuse rather than hallucinate.

1.5.6 Incident investigation and release operations

Agents that parse alerts, search runbooks, correlate logs, and assemble incident summaries or go/no-go release recommendations. These sit at the intersection of coding agents and support agents, and the harness engineering emphasis is multi-tool coordination, trajectory evaluation, and evidence assembly.

1.6 Harness the platform, harness the concept

The word overloads. This book uses *harness engineering* to refer to the general discipline. It uses **Harness** — capitalised — exclusively to refer to the Harness platform (harness.io), a pipeline-native delivery platform that has embraced AI-agent patterns as first-class features: Harness Agents, the Harness DevOps Agent, Harness Skills, and the Harness MCP Server.

Part XI of this book treats the Harness platform in depth as a worked example of a production harness in the general sense. Three reasons for this choice:

1. **It is pipeline-native.** Pipelines are a pre-existing secure control plane. Running agents inside pipelines gives you RBAC, audit, retry, rollback, and notifications *for free*, because the pipeline runtime already provides them. This is the single most instructive pattern in the book.
2. **It exposes an MCP Server.** The Harness MCP Server is a real, production-deployed implementation of the Model Context Protocol. Studying it grounds the MCP chapter (Chapter 7) in a concrete system rather than a specification.
3. **Its Skills catalogue is public.** The Harness Skills are a worked example of the **Skill** abstraction (a versioned, reusable, curated-context template) that Chapter 5 treats abstractly.

If you are interviewing at Harness, Part XI is essential. If you are not, Part XI is still valuable, because it makes the abstract concepts concrete. The generic principles transfer; the specific names do not.

1.7 The shift from prompt engineering to harness engineering

A short recent history, to close out the chapter.

In 2022 and 2023, the prevailing discipline was *prompt engineering*: crafting the exact string that would make the model behave. The craft was real and the outputs were sometimes remarkable, but it did not compose. Prompts were artisanal. They did not version well, did not review well, did not generalise across tasks, and did not survive model upgrades.

In 2024, the focus moved to *agent frameworks*: LangChain, LangGraph, the OpenAI Agents SDK, AutoGen. These frameworks provided a loop, a tool registry, and a notion of state. They

were valuable, but they often shipped as thin wrappers over a while-loop, and the hard problems — evaluation, observability, safety, cost — were left as exercises.

In 2025, the industry began to converge on the vocabulary that this book uses. The reasons were practical. A hundred production incidents, across a hundred companies, turned out to have the same root causes: missing verification, missing sandboxing, missing guardrails, missing traces. The field realised it was rebuilding the same concentric shells around the model in every company, and gave them a collective name.

In 2026, where this book sits, the discipline is recognisable. Senior engineers at frontier labs, platform companies, and AI-first startups share a vocabulary. The interview loops have stabilised around a predictable set of rounds. The patterns have names. The failure modes have taxonomies. The tooling is not yet settled, but the *ideas* are.

The point of this book is to give you those ideas in a form you can use. Structure in, structure out.

Interview Focus

Chapter 1 interview questions.

1. **“Define AI Harness Engineering in one sentence for a non-technical stakeholder.”**

Trap: using jargon. *Frame:* “It’s the engineering around an AI model that makes its outputs safe, auditable, and reliable enough for a business to depend on.” Then offer a one-line analogy (guardrails for a self-driving car; brakes, not the engine).

2. **“What’s the difference between an AI engineer and a harness engineer?”**

Trap: treating the roles as identical. *Frame:* an AI engineer designs the agent’s intelligence layer — prompts, tools, workflow. A harness engineer designs the cross-cutting runtime that every agent depends on — sandboxing, guardrails, evals, observability. The harness engineer’s work is leveraged across many agents; the AI engineer’s work ships one agent at a time.

3. **“Why isn’t a well-prompted model enough for production?”**

Trap: talking about hallucination only. *Frame:* name the six properties — determinism on demand, auditability, permission enforcement, cost bounds, adversarial robustness, and composition — and argue that none of them come from prompting. They come from the harness.

4. **“Walk me through Agent = Model + Harness. What breaks if you remove the harness?”**

Frame: enumerate the loss. No trace $\Rightarrow$ no post-mortem. No sandbox $\Rightarrow$ blast radius is the whole host. No guardrails $\Rightarrow$ PII leaks and injection. No evals $\Rightarrow$ silent regression on model upgrade. No cost controls $\Rightarrow$ a \$4,000 eval run. You are demonstrating that each harness concern is *load-bearing*.

5. **“Describe one production use case and its harness.”**

Frame: pick one of the six categories above you actually know, or have a strong opinion on, and walk through the seven layers (which you will meet in Chapter 3). Interviewers love this question because it separates candidates who have read about agents from candidates who have built one.

CHAPTER 2

Core Agent Concepts

An agent is a loop that has opinions about when to stop.

— Working definition

2.1 The agent loop

Every agent, no matter the framework, is a loop over the same four operations:

observe → plan → act → verify → repeat

Observe. The agent reads the current state: the user’s task, any new tool results from the previous step, any retrieved context, and its own prior reasoning. Observation is the first place failures enter the system; a malicious tool result or a poisoned retrieved document will contaminate every downstream step unless the harness sanitises it here.

Plan. The agent decides what to do next. In the simplest agent (pure ReAct) this is an unstructured thought. In more sophisticated agents (Plan-and-Execute, Reflexion), the plan is explicit and persists across steps. The plan is where interpretability lives; tracing it is essential.

Act. The agent emits a tool call, a structured output, or a final answer. The harness dispatches the call to the appropriate executor (tool server, MCP server, sandbox, LLM judge, and so on), applies input validation and policy checks, and captures the result.

Verify. The agent (or the harness on its behalf) checks whether the act produced the expected effect. This is the loop’s self-correction signal. Verification is the single most under-engineered part of most agent systems — Chapter 12 is devoted to it.

Repeat. The loop continues until a termination condition fires: a final answer, a step-budget exhaustion, a guardrail trigger, an approval gate, or a hard timeout. Termination is a first-class design concern and deserves its own section of the harness.

2.1.1 A minimal agent loop in pseudocode

```
def run_agent(task, tools, max_steps=20, token_budget=100_000):
    state = AgentState.initial(task)
    traces = []

    for step in range(max_steps):
        # OBSERVE
        context = assemble_context(state, retrieved=retrieve(state))
```

```

# PLAN + ACT
response = llm.call(
    system=state.system_prompt,
    messages=context,
    tools=tools.schemas(),
)
traces.append(response.trace)

if response.tool_calls:
    for call in response.tool_calls:
        # Policy gate: guardrails, permission check, dry-run.
        if not policy.allows(call, state):
            return Escalation(reason=policy.reason(call))

        result = tools.dispatch(call, timeout=30)

        # VERIFY (at minimum: did the tool succeed?).
        state.record(call, result)
    else:
        # Final answer path.
        return Completion(answer=response.text, traces=traces)

if state.tokens_used > token_budget:
    return Escalation(reason="token budget exceeded")

return Escalation(reason="max steps exceeded")

```

Listing 2.1: A minimal but honest agent loop. Every production agent is this loop plus a great deal of harness.

Five things to notice about Listing 2.1:

1. Every path that ends the loop returns a structured outcome — **Completion** or **Escalation** — not an exception. Agents that can run out of steps silently are agents that will produce partially-applied changes in production.
2. `policy.allows(call, state)` runs *before* `tools.dispatch(call)`. Guardrails are not an afterthought. They are a gate.
3. The `traces` list collects every LLM call. In a real harness, it collects every tool call, every guardrail event, every retrieval, and every policy decision. See Chapter 17.
4. `token_budget` is enforced in the loop, not hoped for in the prompt. Budgets belong in code.
5. The function’s signature says nothing about models, vendors, or specific frameworks. The harness abstracts all of that. This is the property you are building toward.

2.2 State and continuation

An agent’s state is everything it carries forward from one step to the next. Four categories:

- **Task state.** The user’s original request, any clarifying answers, any constraints.
- **Execution state.** The history of tool calls and their results; the current working directory in a sandbox; open file handles; partial outputs.

- **Cognitive state.** The agent’s own reasoning history — the “thoughts” it has produced, the plans it has formulated, the dead ends it has abandoned.
- **Bookkeeping state.** Token counts, step counts, elapsed time, costs, retries, escalations.

For short tasks, all four fit in the context window. For long tasks, they do not, and the harness must decide what to keep in-context, what to summarise, what to persist externally, and what to retrieve on demand. This is the concern of Chapter 9.

Key Insight

The hardest production bug class is state-shape inconsistency: the agent *thinks* the system is in state s_1 , but it is actually in state s_2 because a previous tool call partially succeeded. Mitigations: idempotent write tools, state re-read after destructive actions, and a verification step that compares observed state to expected state.

2.3 Tools and function schemas

A **tool** is a function the agent can invoke. The model does not execute the function; it emits a structured request (a tool call) and the harness dispatches it. This separation is the entire safety story of tool-using agents.

A tool has three user-visible parts:

1. **Name.** Verb-noun, unambiguous, short. `delete_file`, not `fileManagement`.
2. **Description.** Written for the model, not the human. A sentence or two on what the tool does, when to use it, and any critical caveats.
3. **Parameter schema.** JSON Schema, with types, required fields, enums, and descriptions on every field.

Tool design is a discipline in itself, covered in depth in Chapter 6. The point here is that tools are the agent’s *action vocabulary*. An agent with good tools and a mediocre prompt will outperform an agent with a great prompt and ambiguous tools every time.

```
{
  "name": "delete_file",
  "description": "Delete a single file from the repository. IRREVERSIBLE. Use only after confirming the file exists and is safe to delete. Does not remove from git history.",
  "parameters": {
    "type": "object",
    "required": ["path", "reason"],
    "properties": {
      "path": {
        "type": "string",
        "description": "Repo-relative path to the file. Must not start with '/' or '..'."
      },
      "reason": {
        "type": "string",
        "description": "One-sentence justification. Used for audit trail."
      }
    }
  },
  "dry_run": {
    "type": "boolean",
    "description": "If true, report what would be deleted without deleting."
  }
}
```

```

        "default": true
    }
}
}
}

```

Listing 2.2: Tool schema sketch for a safe file-deletion tool.

Observe the defaults. `dry_run` defaults to `true`, so the agent must actively opt out of safety to perform the destructive action. This is a pattern you will see repeatedly in Chapter 6: make the safe path the easy path.

2.4 Handoffs and specialist sub-agents

A **handoff** is an agent transferring control to a different agent — typically a specialist — for a subtask outside its primary scope. The handoff pattern appears for three reasons:

- **Scope management.** A single agent with fifty tools suffers: the context window fills with schemas and the model’s tool-selection accuracy drops. Splitting into specialists keeps each agent’s surface area tight.
- **Permission isolation.** A code-reading agent should not have code-deletion tools. A handoff forces an explicit trust boundary between the reader and the deleter.
- **Model routing.** A cheap model can classify. An expensive model can plan. Handoffs let each step run on the appropriate model.

Handoffs are simple in theory and treacherous in practice. The three classic failure modes are:

1. **Routing loops.** Agent A hands off to B, which hands back to A, which hands off to B. The harness needs a handoff-depth limit and a recurrence check.
2. **Context loss at the boundary.** The receiving agent does not know the full task context, only the handoff message. Serialising the right amount of context is a design decision.
3. **Authority confusion.** Who is responsible for the final answer — the first agent or the specialist? The handoff protocol must specify.

Handoffs are covered architecturally in Chapter 19.

2.5 Single-agent versus multi-agent: a decision framework

The multi-agent narrative gets more airtime than it deserves. For most tasks, most of the time, a single well-harnessed agent outperforms a multi-agent system on three important metrics: cost, latency, and debuggability. A multi-agent system is *coordination*, and coordination has a tax.

Use a single agent when any of the following apply:

- The task fits comfortably in one context window.
- The required tools all share the same permission scope.
- The failure mode is debuggable by reading a single trace.
- The task does not require parallel exploration.

Consider multi-agent when *two or more* of the following apply:

- Roles are genuinely separable (planner vs. executor; proposer vs. critic).
- Parallel exploration helps (multiple hypotheses explored simultaneously, best-of-N).
- Permission boundaries are load-bearing (a read-only investigator must hand off to a write-capable fixer with an approval step in between).
- Scale exceeds one context window (a large codebase, a long document).
- Different subtasks benefit from different models (classification on a small model, reasoning on a large one).

Common Trap

“Let’s use a supervisor and three workers” is not an answer. It is a design starting point that you now have to defend with specific failure modes, specific coordination costs, and specific benefits. In interviews, justify every additional agent you introduce with a concrete property it unlocks that a single agent cannot provide.

2.6 When not to use an agent at all

Agent hype obscures a quiet truth: for many problems, a deterministic chain is the right answer. *Complexity is a tax*, and agents are the most complex pattern in the LLM toolbox. They get you:

- Non-determinism in control flow.
- Variable latency (can be $10\times$ a direct call).
- Variable cost (can be $50\times$ a direct call).
- An enormous failure surface (every tool call is a possible failure).
- Opaque debugging (the reason the agent chose path A over path B is not always introspectable).

You want agents when the task genuinely requires *iterative* problem solving — when the right next action depends on the result of the previous action in a way that cannot be known at design time. Triage against a decision tree is not that. Translation is not that. Summarisation is not that. Writing code that passes tests, investigating why a build failed, remediating a CVE — these are.

A useful heuristic: *if you can write down the full decision graph in advance, you want a chain, not an agent.*

2.7 Canonical loop architectures

Four loop shapes recur in the literature and in production. Interviewers expect fluency with all four.

2.7.1 ReAct

The simplest loop: interleave reasoning and acting. At each step, the model produces a “Thought:” followed by an “Action:”. The action is executed, the observation is fed back, and the loop repeats.

Strengths. Simple, readable traces, works well for 2–5 step tasks. Most production agents start here.

Failures.

- **Loop on repeated action.** The model calls the same tool with the same arguments three times because nothing is forcing it to reconsider.
- **Premature conclusion.** The model claims it has solved the task after one tool call when the task required several.
- **Action drift.** On long tasks, the model starts inventing actions that are not in the tool list because the tool list has fallen out of attention.

2.7.2 Plan-and-Execute

The loop is split into two phases. First, a planner produces an explicit, enumerated plan (often 3–10 steps). Then an executor works through the plan step by step, possibly with a re-planning step at the end if the results disagree with expectations.

Strengths. Better for long-horizon tasks. The plan is an artefact that can be reviewed, edited, or approved by a human before execution.

Failures.

- **Plan staleness.** The plan was good at step 1 but wrong by step 5 because the world changed. Without re-planning, the executor marches on regardless.
- **Over-planning.** The planner produces 20 steps for a task that needed three. Wasted tokens, wasted time, and more chances to err.
- **Planner/executor disagreement.** The executor encounters an unexpected result and has no protocol for escalating back to the planner.

2.7.3 Reflexion

After each attempt, the agent reflects on what went wrong and stores that reflection in an episodic memory. Subsequent attempts read those reflections as part of context.

Strengths. Genuinely improves on multi-attempt tasks with a clear success signal (code passes tests, math is correct). The reflection becomes a form of self-generated few-shot example.

Failures.

- **Reflection poisoning.** A wrong reflection (the agent misdiagnoses why it failed) makes future attempts worse, not better.
- **Verbosity drift.** Reflections accumulate, context fills, and the agent spends more tokens re-reading its own essays than solving the problem.

2.7.4 Tree-of-Thoughts

The agent explores multiple reasoning paths in parallel, scores them, and commits to the best. In practice this is rarely used directly in production agents; it is more commonly used inside reasoning-heavy subtasks.

Strengths. Can find solutions that a linear search misses. Well suited to puzzles, constraint-satisfaction problems, and code generation where multiple candidate implementations exist.

Failures.

- **Cost.** Exploring four branches to depth five is twenty LLM calls for a task that might have taken three.
- **Scorer brittleness.** The quality of the search depends entirely on the quality of the branch-scoring function. A bad scorer gives you expensive bad answers.

Key Insight

These four are not alternatives you pick from a menu. They are *primitives* you compose. Most real agents are ReAct with a Plan-and-Execute phase at the top and a Reflexion-style memory update at the bottom. Be ready to explain which piece of your architecture is doing which job and why.

2.8 Agent looping failure

The most common failure mode in production agents is not hallucination. It is looping. The agent gets stuck calling the same tool over and over, or alternating between two tools, or re-reading its own output forever. The harness must detect this and break out of it. Three layers of defence:

2.8.1 Layer 1: hard limits (the floor)

Every agent run has three non-negotiable bounds:

- **Step budget** (e.g., 30 tool calls). On exceed, escalate with the trace. Never silently complete.
- **Token budget** (e.g., 100,000 input tokens across the run). On exceed, escalate. This also bounds cost.
- **Wall-clock budget** (e.g., 5 minutes). On exceed, escalate. This bounds user-facing latency.

These are the floor. They catch pathological cases but are coarse. A 30-step agent that repeats the same call for 29 steps still burns budget before it escalates.

2.8.2 Layer 2: behavioural detection

The harness watches the trace for patterns that indicate a stuck agent:

- **Exact-duplicate tool calls.** The same tool with the same arguments, called twice in a row (or N times in the last K calls). Usually a signal to inject a reflection prompt or escalate.
- **Near-duplicate calls.** The same tool with arguments that differ only cosmetically. Requires fuzzy comparison (e.g., string-edit distance on serialised args).
- **No state progress.** Several steps in a row where no externally observable state has changed — no file edited, no ticket updated, no test run. The agent is “thinking” without acting.

When behavioural detection fires, the harness injects a system message (“you have called this tool three times with the same arguments; the result has not changed; reconsider your approach”) or escalates to a human.

2.8.3 Layer 3: structural prevention

The best loop defence is making loops architecturally impossible in the first place:

- **Idempotency keys.** Destructive tools require a key; the second call with the same key is a no-op that returns the original result. Eliminates accidental double-execution.
- **State-changing actions must advance state.** If a write tool returns “nothing changed,” the harness treats that as a weak signal that the agent is misreading the world and may benefit from a re-observation step.
- **Forced exploration.** After N failed attempts at the same approach, the harness prompts the agent to try a different tool or escalate to a human. This is the “if you’ve done the same thing three times, do something else” heuristic, encoded.

Interview Focus

Chapter 2 interview questions.

1. **“Walk me through the ReAct loop. Where does it fail?”**

Frame: state the four operations (observe, plan, act, verify). Name the three classic failures: repeated-action loop, premature conclusion, action drift. Contrast with Plan-and-Execute (better plan artefact, worse plan staleness) and Reflexion (learns across attempts, risks reflection poisoning).

2. **“Your agent enters an infinite loop of the same tool call. Give me three layers of defence.”**

Frame: hard budgets (step/token/wall-clock) as the floor; behavioural duplicate-call detection in the middle; structural idempotency and forced exploration as the ceiling. Note that layer 1 alone is insufficient because it burns budget before firing.

3. **“When would you recommend a simple chain over an agent?”**

Frame: if the decision graph can be drawn in advance, use a chain. Agents pay for non-determinism; pay only when the next action genuinely depends on the previous result.

4. **“Design a handoff protocol between a reader agent and a writer agent.”**

Frame: what context is serialised at the boundary; what the reader’s output contract looks like; how authority flips; how the handoff-depth limit is enforced; where the approval gate sits; who owns the final answer.

5. **“Given a task, how do you decide single-agent vs. multi-agent?”**

Frame: the single-agent defaults (one context, one permission scope, one trace) against the multi-agent triggers (role separation, parallel exploration, permission boundaries, scale, model routing). Insist on at least two triggers before adopting multi-agent.

6. **“What’s the difference between a tool and a handoff?”**

Frame: a tool returns a result to the same agent; a handoff transfers control. Tools share the caller’s context; handoffs serialise a new one. A handoff is typically used when the sub-problem wants its own permission scope or its own model.